

GLOBALRAIN

Practices for Secure Software Report

Table of Contents

DOCUMENT REVISION HISTORY 3
CLIENT 3
INSTRUCTIONS 3
DEVELOPER 4
1. ALGORITHM CIPHER 4
2. CERTIFICATE GENERATION 4
3. DEPLOY CIPHER 4
4. SECURE COMMUNICATIONS 4
5. SECONDARY TESTING 4
6. FUNCTIONAL TESTING 4
7. SUMMARY 4
8. INDUSTRY STANDARD BEST PRACTICES 4

Document Revision History

Version	Date	Author	Comments
1.0	[10/17/2026]	[Dashayla Fields]	

Client



Instructions

Submit this completed practices for secure software report. Replace the bracketed text with the relevant information. You must document your process for writing secure communications and refactoring code that complies with software security testing protocols.

- Respond to the steps outlined below and include your findings.
- Respond using your own words. You may also choose to include images or supporting materials. If you include them, make certain to insert them in all the relevant locations in the document.
- Refer to the Project Two Guidelines and Rubric for more detailed instructions about each section of the template.

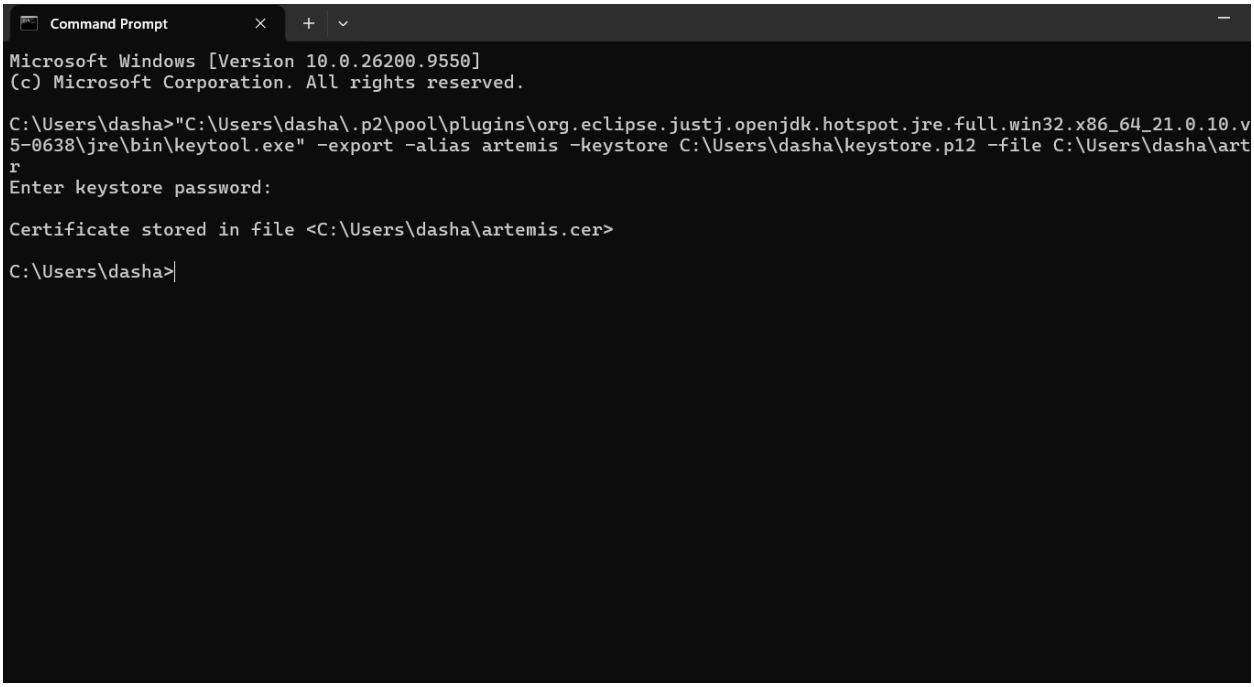
Developer
[Dashayla Fields]

1. Algorithm Cipher

For this project, SHA-256 was chosen as the hashing algorithm because it provides a reliable way to verify that information has not been altered during transmission. The algorithm creates a unique 256-bit hash value that acts like a digital fingerprint for the data. Even a small change to the original input will produce a completely different hash result, making it useful for validating data integrity. SHA-256 is part of the SHA-2 family and is widely trusted across the technology industry due to its strong security and resistance to hash collisions. Unlike encryption methods that use symmetric or asymmetric keys to protect data, SHA-256 is a one-way hashing function designed for verification rather than data recovery. In this project, it was implemented to generate checksums that help confirm data remains unchanged while being transferred. Its widespread adoption and proven security make it a practical choice for protecting sensitive information within the Artemis Financial application.

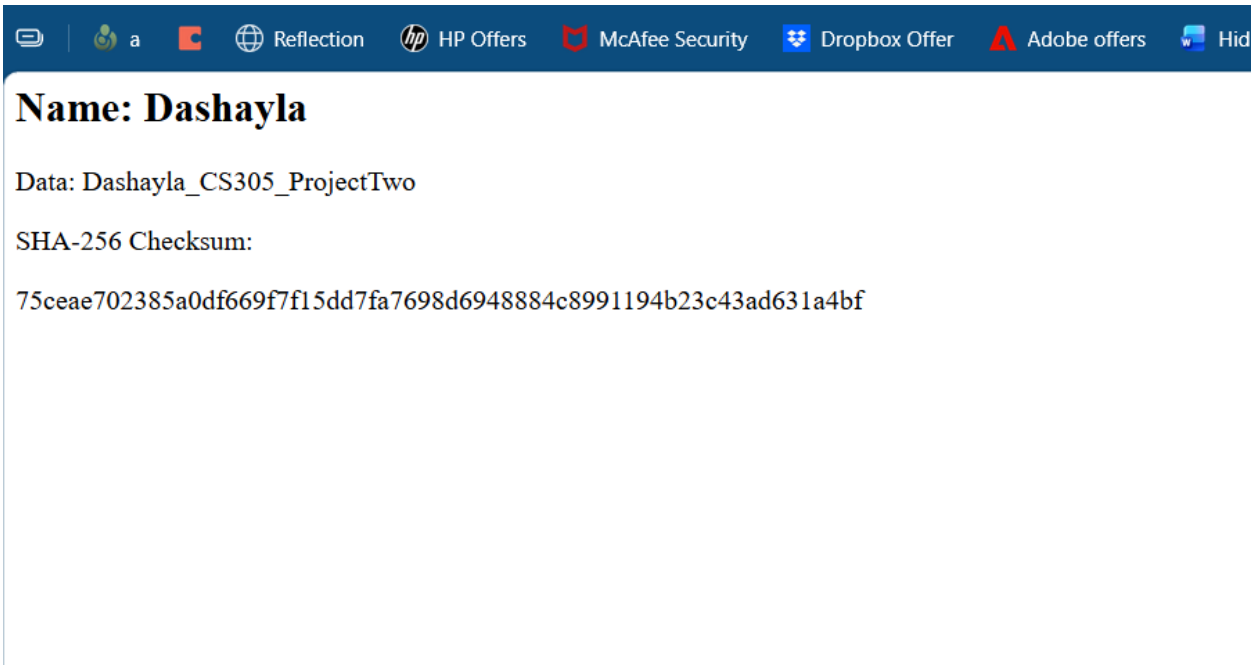
2. Certificate Generation

Insert a screenshot below of the CER file.



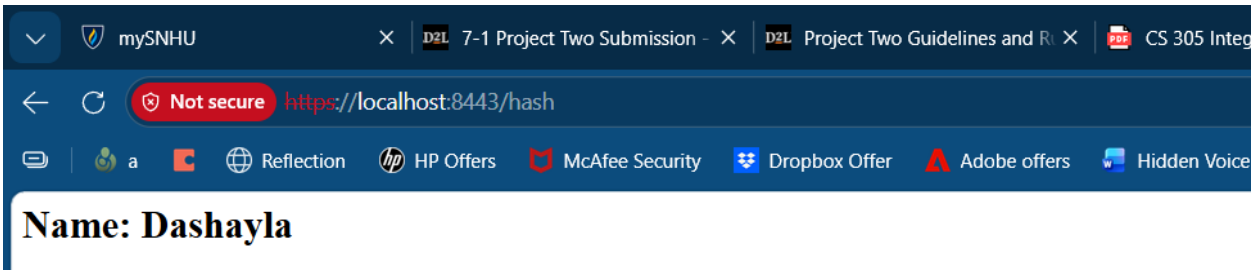
3. Deploy Cipher

Insert a screenshot below of the checksum verification.



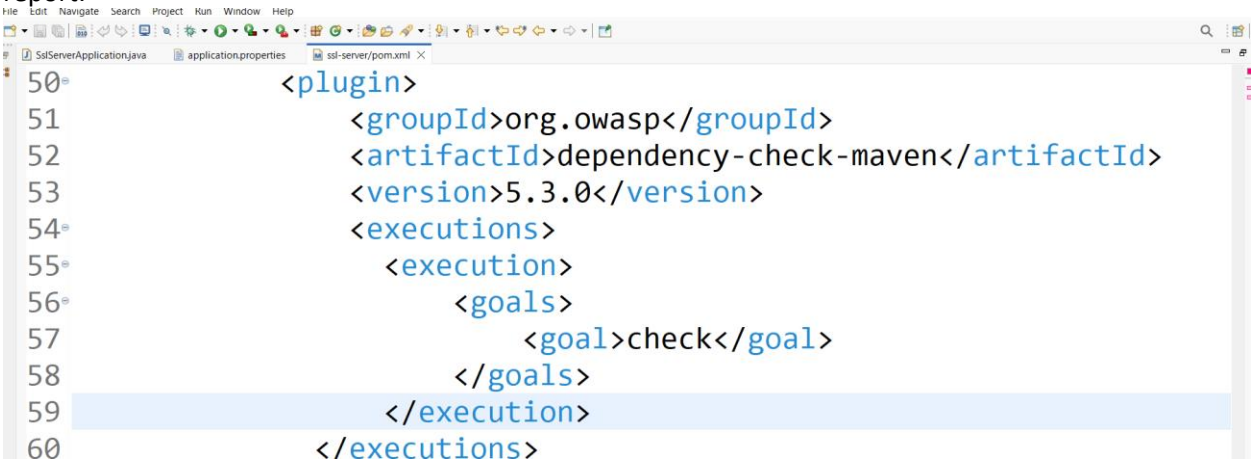
4. Secure Communications

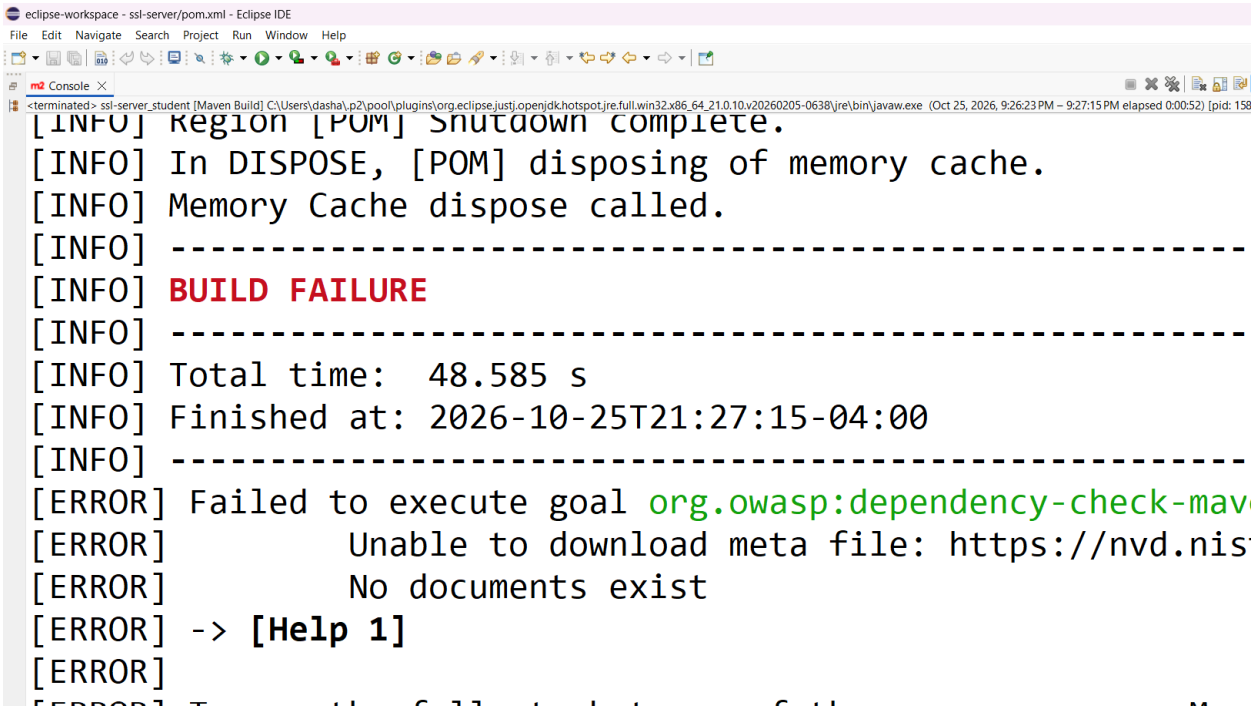
Insert a screenshot below of the web browser that shows a secure webpage.



5. Secondary Testing

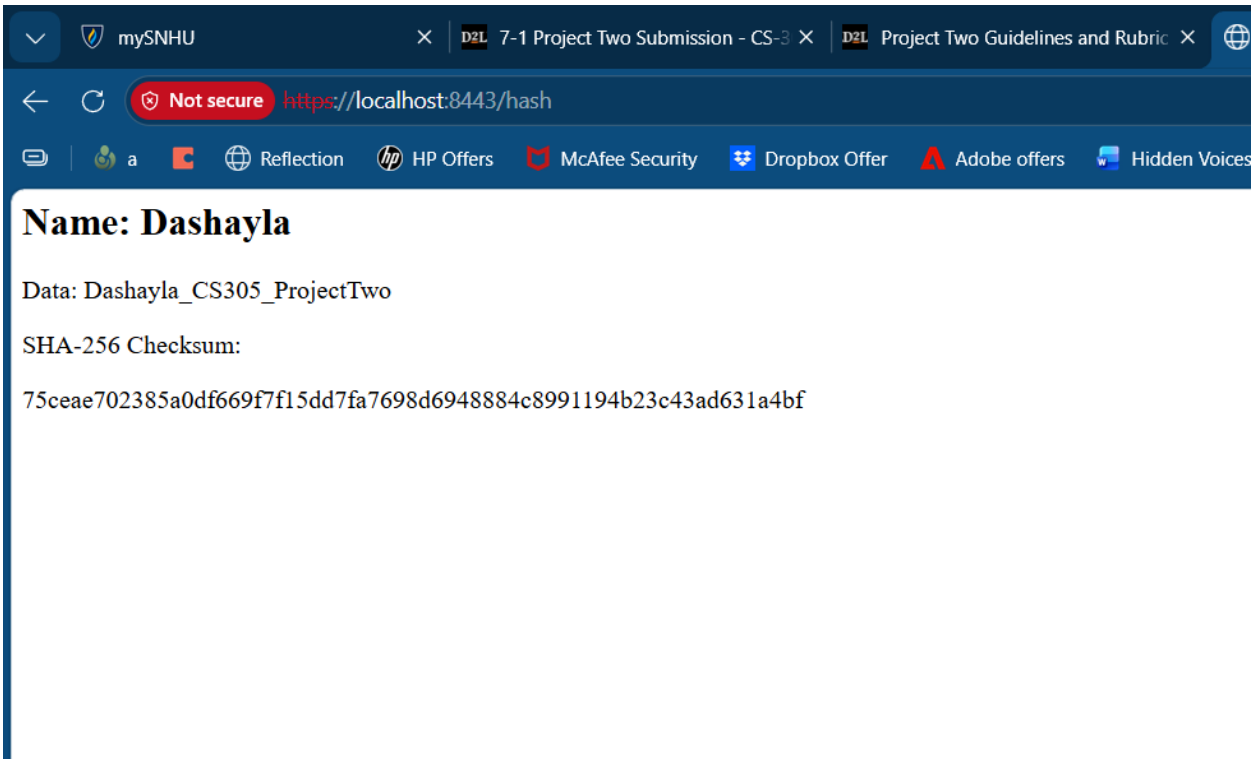
Insert screenshots below of the refactored code executed without errors and the dependency-check report.





6. Functional Testing

Insert a screenshot below of the refactored code executed without errors.



7. Summary

The Artemis Financial application was refactored to improve the overall security of the system and better protect sensitive financial information. Throughout this project, several security enhancements were added, including secure HTTPS communication, SSL certificate configuration, and SHA-256 checksum verification. These updates help reduce potential security risks by encrypting data transmitted between the client and server while also providing a way to verify that transferred data has not been altered.

During the refactoring process, multiple layers of security were incorporated into the application. A self-signed SSL certificate was generated and configured to enable HTTPS communication, helping protect data while it is being transmitted over the network. Additionally, a SHA-256 hashing function was implemented to generate checksums that can be used to validate data integrity and detect unauthorized changes. Security testing was also performed to verify that the application continued to function correctly after the enhancements were applied. The OWASP Dependency-Check plugin was integrated into the project to support the identification of known vulnerabilities within project dependencies as part of the application's overall security review process.

Completing this project demonstrates how multiple security practices can work together to strengthen an application. Rather than relying on a single safeguard, the project combines secure communication, data validation, and vulnerability analysis to create a more secure environment. These improvements help protect customer information, support secure software development

practices, and provide Artemis Financial with a stronger foundation for maintaining the security and reliability of its application moving forward.

8. Industry Standard Best Practices

Industry-standard best practices were applied throughout this project to strengthen the security of the Artemis Financial application while maintaining its existing functionality. HTTPS was implemented to encrypt data transmitted between the client and server, helping protect sensitive financial information from unauthorized access. A SHA-256 hashing algorithm was also added to generate checksums and verify data integrity, ensuring that information remains unchanged during transmission.

Additional security measures included generating and configuring a digital certificate and performing dependency analysis using the OWASP Dependency-Check plugin. These practices help reduce potential security risks and support a more secure software development process. By following accepted security standards and incorporating multiple layers of protection, the application is better equipped to safeguard customer information and maintain the reliability and trustworthiness of the system.

Applying secure coding best practices provides long-term value to the organization by helping prevent data breaches, reducing security vulnerabilities, and increasing user confidence in the software. Taking a proactive approach to security not only protects sensitive data but also contributes to the overall success and reputation of the company.